

Nuclear Winter

A Developer's Learning Guide — Coding, DevOps & System Design

Built around the Nuclear Winter game · Documentation for junior developers

2026

Contents

Nuclear Winter — Developer's Learning Guide	2
The three lenses	2
Suggested reading order	2
How to use this guide	3
A note on scope and honesty	3
01 — System Design	4
1. The 10,000-foot view	4
2. The files, and why there are six of them	4
3. The module pattern: how six files avoid stepping on each other	6
4. The state machine: the game's brain	7
5. Data flow: one frame, start to finish	8
6. How the subsystems collaborate (without tangling)	9
7. What you'd do differently at scale (and why it's fine here)	9
02 — Code Walkthrough	11
Reading order inside a file	11
data.js — start here, it's the easiest	11
game.js — the engine	12
combat.js — a self-contained subsystem	15
maps.js — a tiny builder DSL	16
Cross-cutting habits you should steal	17
03 — Graphics & Audio (from nothing but code)	18
Part 1 — Graphics: drawing pixels with code	18
Part 2 — Audio: sound as pure math	21
Why generate assets instead of loading files?	23
04 — DevOps	24
1. The lifecycle of a change	24
2. The inner loop: edit → run → observe	24
3. The outer loop: version control with Git	25
4. Deploying: getting it onto the internet	26
5. Automation: CI/CD, and a starter pipeline	27
6. The mental model to keep	28
05 — Exercises & Extension Roadmap	29
Tier 0 — Orientation (find your footing)	29
Tier 1 — Tuning (change data, not logic)	29

Tier 2 — Small logic changes (touch the engine gently)	29
Tier 3 — New features (design a little)	30
Tier 4 — Architecture & Ops (think like a builder)	30
How to debug when it breaks (it will)	31
Where to go after this codebase	31

Nuclear Winter — Developer’s Learning Guide

Welcome. This folder is not just “how the game works” — it’s a **teaching guide** built around a real, complete codebase. If you’re a junior developer trying to level up in **coding**, **DevOps**, and **system design**, the idea is simple: you learn those three things best by reading real code that already works, understanding *why* it’s shaped the way it is, and then changing it yourself.

Nuclear Winter is a good specimen to learn from because it’s:

- **Small enough to hold in your head** — ~2,500 lines across 6 JavaScript files.
- **Complete** — title screen, save system, combat, quests, an ending. Nothing is faked.
- **Zero-dependency** — no framework, no build step, no `node_modules`. Every line is code you can read. Nothing is hidden behind a library.
- **Honest about its patterns** — it uses the same ideas (game loop, state machine, data-driven design, module boundaries) that scale up to much bigger software.

The three lenses

You’ll see the same codebase explained through three different lenses. They overlap on purpose — that overlap *is* the lesson. Good code, good ops, and good design are not separate activities; they’re the same decisions viewed from different distances.

Lens	Question it asks	Where in these docs
Coding	<i>How is this line written, and why that way?</i>	02-code-walkthrough
System design	<i>Why is the code split into these pieces?</i>	01-system-design
DevOps	<i>How does it get from my editor to a player?</i>	04-devops

Suggested reading order

1. ****01 — System Design**** — the map before the territory. How the whole thing fits together, the module boundaries, the state machine that drives everything. Read this first even though it’s the most abstract — the rest makes more sense once you have the shape in your head.
2. ****02 — Code Walkthrough**** — the territory. A guided tour of each file, the patterns used (the game loop, delta time, collision, y-sorting, the module pattern, the save system), with real excerpts and the reasoning behind them.
3. ****03 — Graphics & Audio**** — the “how did they make art and sound with no asset files?” chapter. Canvas drawing, seeded random number generators for procedural art, and synthesizing sound from math with Web Audio.
4. ****04 — DevOps**** — how code becomes a running thing other people can use. Version control, the “no build step” philosophy and its trade-offs, serving, deploying to the web for free, and a starter CI pipeline.

5. ****05 — Exercises**** — you don't learn by reading, you learn by changing things and watching them break. Graded exercises from “change a number” to “design a new subsystem,” plus how to debug when it goes wrong.

How to use this guide

- **Keep the code open next to the docs.** Every reference like `js/game.js` is a real file. When a doc quotes a function, go find it in the source and read the lines around it.
- **Run it constantly.** Instructions are in the top-level **README**. Change something small, refresh the browser, see what happened. That loop — edit, run, observe — is the single most important habit in this whole guide.
- **Don't try to memorize.** The goal is to recognize patterns. The next time you meet a “game loop” or a “state machine” or a “seeded RNG” in a different codebase, you'll think *oh, I've seen this shape before*. That recognition is the whole point.

A note on scope and honesty

This is a learning codebase, not production software for a team of 50. Some choices here (global IIFE modules, `localStorage` saves, no automated tests) are perfectly right for a solo browser game and would be wrong for a banking backend. Where that's true, the docs say so and point at what you'd do instead at scale. Learning to tell those two situations apart is more valuable than any single technique.

Created by Victory Ma · Documentation for learners.

01 — System Design

System design is the art of deciding where to draw the lines. Before a single line is written, someone decides what the pieces are, what each piece is responsible for, and how they're allowed to talk to each other. Get those lines right and the code almost writes itself. Get them wrong and every change fights you.

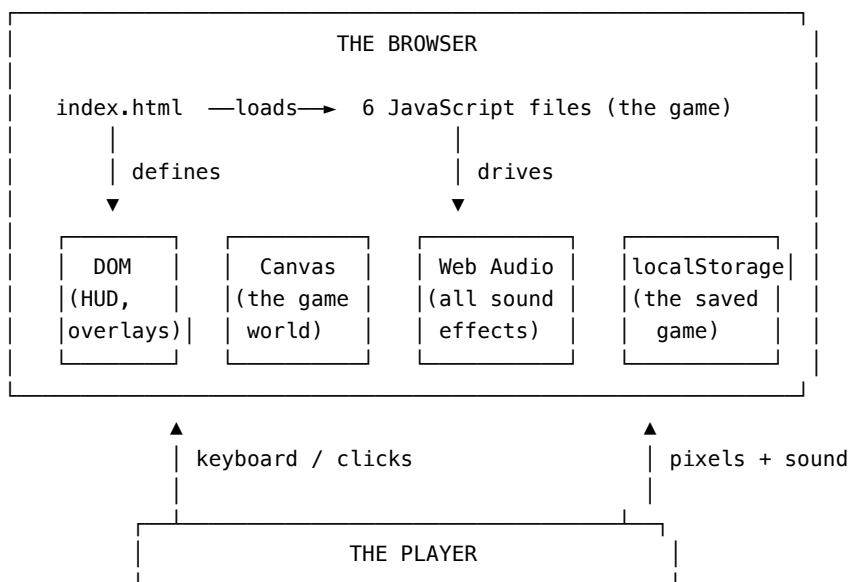
This chapter is about those lines. We'll look at Nuclear Winter from far away first, then zoom in on the two ideas that hold it together: **module boundaries** and the **state machine**.

1. The 10,000-foot view

Nuclear Winter is a **browser game**. That single fact decides a huge amount:

- It runs entirely on the player's machine (the "client"). There is **no server** doing game logic, no database, no network calls during play.
- Its "platform" is the browser's three built-in engines: the **DOM** (page structure), **Canvas** (a 2D drawing surface), and **Web Audio** (a sound synthesizer). Everything the game does is orchestrating those three.
- Its "storage" is the browser's `localStorage` — a tiny key-value box that survives page reloads.

Here's the whole system on one diagram:



Notice what's **not** there: no backend, no build tools, no external services. This is the simplest possible shape a piece of interactive software can have, which is exactly why it's a good place to learn. Every arrow on that diagram is something you can point at in the code.

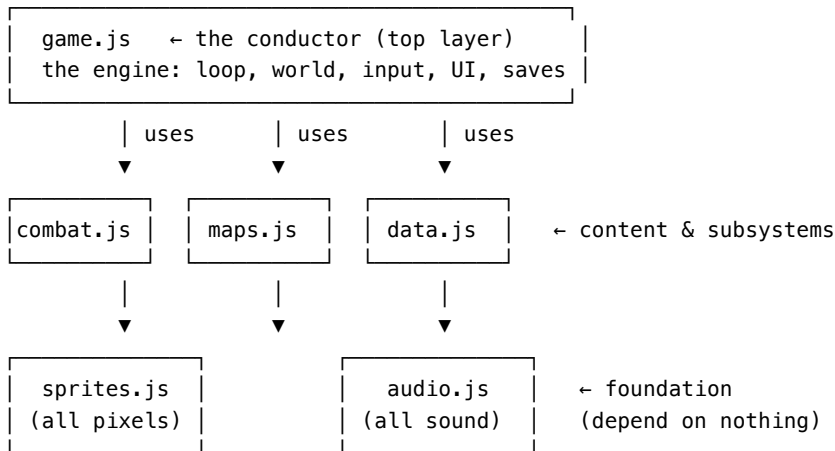
2. The files, and why there are six of them

Open `index.html` and look at the bottom:

```
<script src="js/audio.js"></script>
<script src="js/sprites.js"></script>
<script src="js/data.js"></script>
```

```
<script src="js/maps.js"></script>
<script src="js/combat.js"></script>
<script src="js/game.js"></script>
```

Six files, loaded in that order. That order is not an accident — **it's the dependency graph made physical**. A file can only use things defined in files loaded *before* it. So the order tells you the layers:



Read that from the bottom up:

File	Responsibility	Depends on
audio.js	Turn math into sound. Knows nothing about the game.	nothing
sprites.js	Turn code into pixel art. Knows nothing about the game.	nothing
data.js	The <i>nouns</i> : weapons, classes, enemies, items, story text.	nothing
maps.js	The <i>places</i> : how each area's tile grid is built.	(uses sprite tile names, but by string)
combat.js	The <i>battle rules</i> : turns, damage, win/lose.	data, sprites, audio
game.js	The <i>conductor</i> : the loop, movement, dialogue, quests, saves, UI.	everything

The single most important design rule here

Dependencies point downward, never upward. `audio.js` never reaches up into `game.js`. `sprites.js` doesn't know `combat` exists. The foundation layers are *ignorant* of the layers above them.

Why does this matter so much? Because it means the lower layers are **reusable and testable in isolation**. You could lift `audio.js` out of this game and drop it into a completely different project unchanged, because it has no idea it's in "Nuclear Winter." That property — *low-level code doesn't depend on high-level code* — is one of the most important ideas in all of software. In big systems it has a name (the **Dependency Inversion Principle**), but the instinct is the same at every scale: **the thing that changes rarely should not depend on the thing that changes often**.

Weapons stats change often (balance tweaks). The sound engine changes rarely. So `data.js` may depend on ideas from `audio.js`, but never the reverse. If you ever feel tempted to make `audio.js` "just quickly check

the player's health," stop — that arrow points the wrong way, and it's the first crack that eventually makes a codebase impossible to change.

Data vs. code

Look at the split between `data.js` / `maps.js` and everything else. This is **data-driven design**, and it's worth naming explicitly.

`data.js` contains no behavior — it's just facts:

```
survivalist: { name: 'Survivalist', maxHealth: 200, atk: 25, def: 20, spd: 5, weapon: 'rifle', ... }
```

`game.js` and `combat.js` contain behavior, but they don't hard-code any specific weapon or enemy — they read from `data.js`. This separation means **you can change the game without changing the engine**. Want a new enemy? Add an object to `enemyTypes`. You wrote no new logic; you added a fact, and the existing logic picks it up. Designers (or you, wearing your designer hat) can tune the whole game by editing data, and the risky engine code stays untouched. Every big game engine, from tiny ones to Unreal, is built on this same divide.

3. The module pattern: how six files avoid stepping on each other

All six files load into the *same* global browser scope. Without discipline, they'd collide — two files both defining a variable called `x` would clobber each other. The game avoids this with the **IIFE module pattern** (Immediately Invoked Function Expression). Every file looks like this:

```
const Game = (() => {  
  // ...lots of private variables and functions...  
  let player = { ... }; // private - nobody outside can touch this  
  function update(dt) { ... } // private  
  
  return { // the public "API" - the only doors in  
    get player() { return player; },  
    updateHUD, gainRewards, addItem, /* ... */  
  };  
})();
```

Unpack that:

- `((() => { ... }()))` defines a function and immediately calls it. Everything inside is in its own private scope.
- Whatever the function returns becomes the module's **public interface**. Here, the outside world gets `Game.player`, `Game.updateHUD()`, etc. — and *nothing else*.
- Everything not returned (like the internal update loop) is **private**. It literally cannot be reached from another file.

So each file exposes exactly one global: `AudioSys`, `Sprites`, `DATA`, `Maps`, `Combat`, `Game`. Six names in the global scope, each a clean container. This is the 1990s–2010s browser answer to “how do I make modules without a module system.” Modern projects use `import/export` (ES modules) instead — see the DevOps chapter for why this project deliberately doesn't — but the *goal* is identical: **hide your internals, expose a small deliberate surface**.

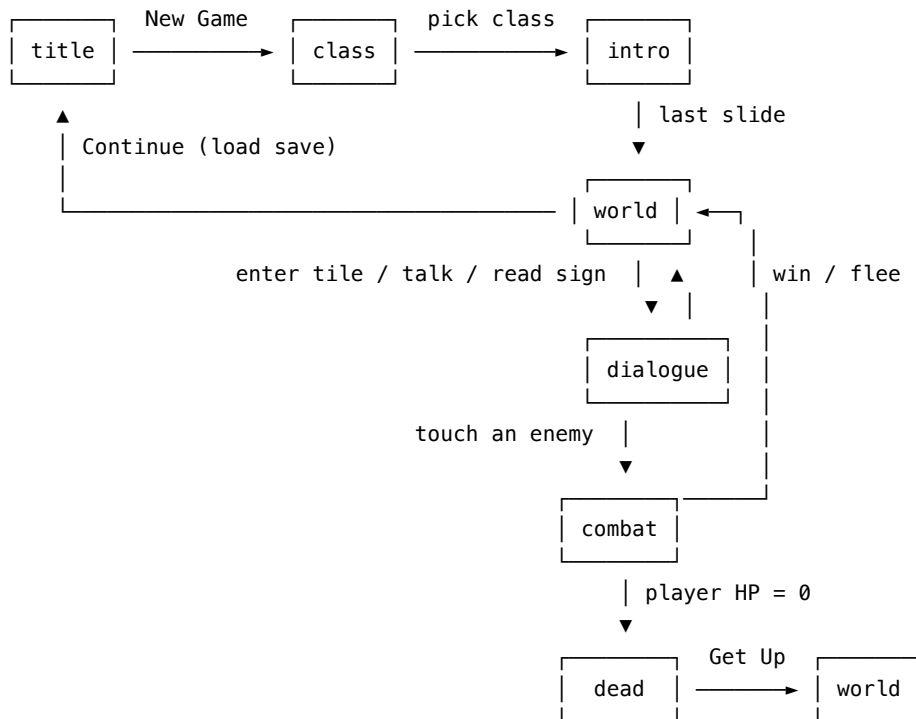
The lesson that outlives the syntax: a module's value is as much in what it *hides* as what it shows. `Combat` exposes `start()` and `bindUI()`. It hides the dozen helper functions and the animation state that make combat work. Someone using `Combat` needs to know “call start with an enemy type”; they do not need to know how the muzzle flash is timed. **A good interface is a small one.**

4. The state machine: the game's brain

Here's the most important runtime concept in the whole codebase. At any instant, the game is in exactly **one** of a handful of *states*, and that state decides what input does and what gets drawn. In `game.js`:

```
let state = 'title'; // title | class | intro | world | dialogue | combat | dead | ending
```

That one variable is the game's brain. This is a **finite state machine (FSM)**: a finite list of states, with defined rules for moving between them.



Why is this such a big deal? Because **without** a state machine, you get a mess of boolean flags: `isInCombat`, `isTalking`, `isPaused`, `isDead`... and then bugs like “the player can walk around *during* a cutscene” because you forgot to check one of five booleans in the movement code. With a single `state` variable, the rule is simple and enforced in one place. Look at how movement is gated in the update function:

```
function update(dt, ts) {
  // ...ambient effects run in every state...
  if (state !== 'world') return; // ← the ONE gate. No movement unless in 'world'.
  // ...player movement, enemy AI, portals, triggers...
}
```

One line. If we're not in the `world` state, movement code simply doesn't run. Dialogue can't be walked out of; combat can't be strolled away from — not because we remembered to check a flag everywhere, but because the whole category of behavior is switched off by that single `return`. The rendering side does the same thing: `render` looks at `state` and draws the title scene, or the world, accordingly.

Input routing works the same way — `onKeyDown` in `game.js` reads `state` and decides what a keypress means:

```
if (state === 'combat') { Combat.handleKey(k); return; } // '1' = attack
if (state === 'dialogue') { /* advance the text */ return; }
if (state !== 'world') return; // title/intro ignore movement
// ...only here does 'E' mean "interact", 'I' mean "inventory", etc.
```

The same physical key press means different things in different states, and the state machine is what makes that clean instead of chaotic.

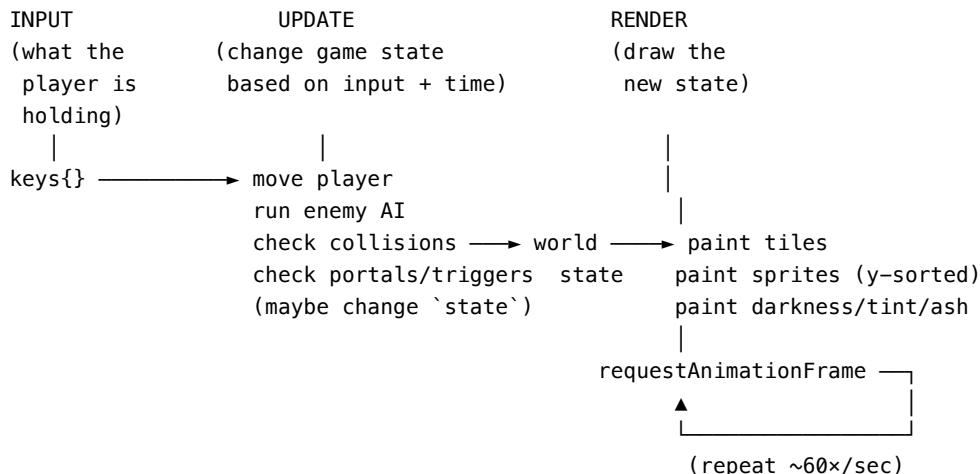
Where you'll meet this again: state machines are *everywhere* once you can see them. A checkout flow (cart → shipping → payment → confirmation). A network connection (connecting → open → closing → closed). A CI pipeline (queued → running → passed/failed). Any time something has “modes,” an explicit state variable beats a pile of booleans.

5. Data flow: one frame, start to finish

System design isn't only about static structure — it's about how data *moves* while the program runs. A game is the clearest possible example because it does the same cycle 60 times a second. That cycle is the **game loop**, and it lives at the bottom of `game.js`:

```
function loop(ts) {  
  const dt = Math.min(0.05, (ts - lastTs) / 1000 || 0.016); // seconds since last frame  
  lastTs = ts;  
  update(dt, ts); // 1. change the world  
  render(ts); // 2. draw the world  
  requestAnimationFrame(loop); // 3. ask the browser to call us again next frame  
}
```

Every frame is two phases, always in this order:



The golden rule, which this loop follows and which you should carry everywhere: **update and render are separate phases**. First you figure out what the world *is* now (update). *Then* you draw it (render). You never draw halfway through changing things, because that produces flicker and tearing. “Compute the new state fully, then display it” is a principle far bigger than games — it’s why UIs use a “virtual DOM,” why databases have transactions, why good systems separate *deciding* from *doing*.

Why `dt` (delta time) matters — a real system-design decision

Look again at `dt` — the number of seconds since the last frame. Every movement in the game is multiplied by it:

```
const spd = 85 * dt; // 85 pixels *per second*, scaled to this frame's slice of time
```


Here's the design problem it solves. Frames don't arrive at a fixed rate — a fast gaming PC might render 144 per second, a tired laptop 30. If we moved the player “3 pixels per frame,” the character would sprint on the fast machine and crawl on the slow one. **The game's difficulty would depend on the player's hardware.** That's a bug baked into the architecture.

By expressing movement as “85 pixels *per second*” and multiplying by `dt`, the player covers the same ground per real-world second *regardless of frame rate*. Fast machine: many small steps. Slow machine: fewer bigger steps. Same destination. This is called being **frame-rate independent**, and deciding to build it in from the start (rather than discovering the bug later on someone else's computer) is exactly the kind of early structural call that “system design” means.

(Note the `Math.min(0.05, ...)` clamp: if the tab is backgrounded and `dt` would be huge, we cap it so the player can't teleport through walls after you switch back. Small line, real robustness thinking.)

6. How the subsystems collaborate (without tangling)

Let's trace one real interaction to see the boundaries doing their job. **The player walks into an enemy.** Follow the responsibility as it hands off:

1. **game.js** — `update()` detects the collision (player and enemy overlap) and calls `startCombat(enemy)`.
2. **game.js** flips `state = 'combat'` and calls `Combat.start(typeId, options)`. Crucially, it passes **callbacks**: `onEnd`, `onLose`. It's saying “*run the fight; here's what to do when you're done — I don't want to know how you run it.*”
3. **combat.js** takes over entirely: it reads the enemy's stats from `DATA`, draws battle scenes with `Sprites`, plays gunfire with `AudioSys`, runs the turn loop. `game.js` is not involved in any of that.
4. When the fight ends, `combat.js` calls the callback `game.js` gave it. Now `game.js` decides the *consequences* — award XP, remove the defeated enemy, maybe trigger a story beat, save the game.

See the clean seam? **Combat knows how to fight. The game knows what fighting means.** They meet at a tiny interface — `start()` plus two callbacks. This pattern (hand a subsystem a job plus “call me back when you're done”) is called a **callback** or **continuation**, and it's how you keep two complex things from growing into each other. `Combat` could be rewritten from turn-based to real-time and `game.js` wouldn't change a line, as long as it still calls `onEnd` when it's over.

The takeaway for design: when two parts must cooperate, spend your effort making the *seam between them* as small and explicit as possible. The parts can be messy inside; the interface between them should be clean. Most “unmaintainable” codebases aren't bad because any one function is bad — they're bad because everything reaches into everything else and there are no seams left to cut along.

7. What you'd do differently at scale (and why it's fine here)

Part of learning system design is calibration: knowing when a “rule” applies and when it doesn't. Here are honest trade-offs this project makes:

This project does...	At bigger scale you'd...	Why it's right <i>here</i>
Global IIFE modules	ES modules or a bundler	6 files, one author, no name clashes in practice.
One big <code>game.js</code> (~900 lines)	Split into movement, dialogue, quests, UI files	Still fits in your head; splitting adds ceremony without payoff yet.
<code>localStorage</code> for saves	A real database + accounts	The save is one player's local progress; nobody else needs it.

This project does...	At bigger scale you'd...	Why it's right <i>here</i>
No automated tests	Unit + integration test suites	You can eyeball the whole game in 10 minutes; a payments system you cannot. There's no shared state between players to coordinate.
No server at all	API + backend	

None of these are “wrong.” They’re *proportionate*. The skill isn’t “always use the biggest hammer” — it’s matching the structure to the problem’s actual size and risk. A junior engineer over-engineers a to-do app into microservices; a senior engineer writes 200 honest lines and moves on. Learning to feel that difference is worth more than any framework.

Next: 02 — Code Walkthrough, where we drop from the map down into the territory and read the actual functions that implement all of this.

02 — Code Walkthrough

The last chapter drew the map. This one walks the territory. We'll read the actual code, file by file, and stop at each pattern worth knowing. The goal isn't to memorize this codebase — it's to recognize these *shapes* when you meet them again somewhere else.

Keep the source open beside this. Every heading points at a real file.

Reading order inside a file

A useful habit when you open any unfamiliar file: **read it outside-in**.

1. First, what does it *export*? (For our modules, scroll to the `return { ... }` at the bottom — that's the public API, the whole point of the file.)
2. Then, what are its main *data structures* near the top?
3. Only *then* dig into individual functions, and only the ones you care about.

You almost never need to read a file top-to-bottom like a novel. Find the surface, then dive where you have a question.

data.js — start here, it's the easiest

data.js is the gentlest file: almost no logic, just carefully organized facts. Read it first to learn the game's vocabulary.

```
const DATA = (() => {
  const weapons = {
    fists: { name: 'Fists',      dmg: 5,  dex: 5,  rng: 5,  icon: 'icon_fists', sound: 'melee' },
    rifle: { name: 'Hunting Rifle', dmg: 15, dex: 5,  rng: 10, icon: 'icon_rifle', sound: 'rifle' },
    // ...
  };
  // items, classes, enemyTypes, shopStock, introSlides, questText ...
  return { weapons, items, classes, enemyTypes, shopStock, introSlides, questText };
})();
```

Two coding lessons hide in this simple file:

1. Reference by name, not by copy. A weapon carries `icon: 'icon_rifle'` and `sound: 'rifle'` — *strings*, not the actual sprite or sound. The real sprite lives in `sprites.js`, the real sound in `audio.js`. `data.js` just holds a *label* that other systems look up. This is **loose coupling**: `data.js` doesn't need to import graphics or audio; it only needs to agree on names. If you rename a sprite, you fix it in two string-places, not by untangling object references. (Databases do exactly this with “foreign keys” — a row points at another row *by id*, not by embedding it.)

2. Objects-of-objects keyed by id. Notice `weapons` is an object like `{ fists: {...}, rifle: {...} }`, not an array `[ {...}, {...} ]`. That means code can say `DATA.weapons.rifle` OR `DATA.weapons[someId]` — an instant, readable lookup by name. Compare the original version of this game, which used arrays and referred to `weapons[1]` — a “magic number” you had to decode. **Naming things beats numbering them.** When you find yourself writing `array[3]` and having to remember what 3 is, reach for a keyed object instead.

game.js — the engine

This is the big one (~900 lines) and the conductor of everything. We won't read all of it; we'll hit the load-bearing patterns.

The player and the flags: your two “state” objects

```
const player = {
  x: 0, y: 0, dir: 'down', frame: 0, moving: false,
  xp: 0, level: 0, gold: 0,
  health: 100, maxHealth: 100, atk: 10, def: 10, spd: 10,
  inventory: [], currentWeapon: DATA.weapons.fists,
  classId: 'none', className: 'None'
};

const flags = {
  quest: 0, enteredTown: false,
  opened: {}, // which chests are looted, by id
  defeated: {}, // which unique enemies are dead, by id
  websBurned: false, factoryUnlocked: false
};
```

There are two kinds of state here, and separating them is deliberate:

- player is **who you are right now** — stats, position, pack. It changes constantly.
- flags is **what has happened in the story** — irreversible world facts. Did you open that chest? Kill that boss? Burn those webs?

Why split them? Because they have different *lifetimes and meanings*. flags is what makes the world remember you. When you loot a chest, flags.opened[id] = true, and forever after the render code draws it open and the interaction code refuses to give loot twice. This is the difference between a world that *reacts* and one that resets — and it's just a plain object used as a set of “things that are true now.”

Pattern: an object as a set. flags.opened = {} then flags.opened['cityKey'] = true. Checking if (flags.opened[id]) asks “has this happened?” Using an object's keys as a set of ids is a cheap, everywhere-useful trick.

The game loop and delta time (the heart)

We met this in chapter 1, but here's the coding detail:

```
function loop(ts) {
  const dt = Math.min(0.05, (ts - lastTs) / 1000 || 0.016);
  lastTs = ts;
  update(dt, ts);
  render(ts);
  requestAnimationFrame(loop);
}
```

requestAnimationFrame(loop) is the browser saying “I'll call your function right before I next paint the screen, and I'll pass you a timestamp.” By calling it again at the end of loop, we create a self-sustaining cycle — the game's heartbeat. Two details worth internalizing:

- (ts - lastTs) / 1000 converts milliseconds-between-frames into **seconds**, so all our speeds can be written in friendly units like “85 pixels per second.”
- || 0.016 is a guard for the *very first frame*, when lastTs is 0 and the math would be nonsense. 0.016 ≈ one 60fps frame. Small defensive touches like this are the difference between code that works in the

demo and code that works.

Movement + collision: the “feet hitbox”

Reading input into a direction is straightforward:

```
let dx = 0, dy = 0;
if (keys.ArrowUp || keys.w) dy = -1;
else if (keys.ArrowDown || keys.s) dy = 1;
if (keys.ArrowLeft || keys.a) dx = -1;
else if (keys.ArrowRight || keys.d) dx = 1;
```

The interesting part is collision. A sprite is 16×16 pixels, but the character’s *body* is drawn in the top two-thirds and their *feet* at the bottom. If we blocked movement whenever any part of the 16×16 box touched a wall, you couldn’t stand with your head “in front of” a tree you’re really standing below. So collision only checks the **feet**:

```
// feet hitbox: 10px wide, 6px tall at the bottom of the 16px sprite
function blocked(x, y) {
  return solidAt(x + 3, y + 10) || solidAt(x + 12, y + 10) ||
    solidAt(x + 3, y + 15) || solidAt(x + 12, y + 15);
}
```

Four points — the corners of a little rectangle around the feet. If any of them lands on a solid tile, that move is blocked. This is a simplified **AABB** (axis-aligned bounding box) collision check, and the “only the feet collide” trick is standard in top-down games because it matches how depth *looks*. The lesson: **collision shape and visual shape are allowed to differ**, and choosing the collision shape is a design decision about how the game should *feel*.

There’s a second subtlety — axes are resolved *separately*:

```
moveEntity(player, dx * spd, 0); // try horizontal
moveEntity(player, 0, dy * spd); // then vertical, independently
```

Why? So that if you’re walking diagonally into a wall, the blocked axis stops but the free one keeps going — you *slide along* the wall instead of sticking to it. Try changing it to move both at once and you’ll immediately feel how much worse it is. This is a great example of a two-line change that transforms game *feel*.

Rendering with a camera and depth sorting

The world is bigger than the screen, so we draw through a **camera** that follows the player and clamps to the map edges (so you never see past the border):

```
cam.x = Math.max(0, Math.min(world.w * TILE - VIEW_W, player.x + 8 - VIEW_W / 2));
```

Read that inside-out: “center the camera on the player (player.x – halfscreen), but never less than 0 and never more than (map width – screen width).” That Math.max(0, Math.min(limit, value)) sandwich is the **clamp** idiom — pinning a number into a range — and you’ll write it a thousand times in your career.

Then the clever bit: **y-sorting for fake depth**. In a top-down view, something “lower” on the screen should appear *in front of* something higher. So we gather every drawable thing into one list, sort by their y position, and draw back-to-front:

```
const draws = [];
// ...push chests, props, npcs, enemies, and the player, each with its y...
draws.sort((a, b) => a.y - b.y); // lowest y first (drawn first = behind)
draws.forEach(d => ctx.drawImage(d.spr, ...));
```

This is the **painter's algorithm**: draw far things first, near things last, let later paint cover earlier paint. It's how you get a 2D game to *look* like it has depth — walk above a barrel and you're behind it; walk below and you're in front. One `.sort()` buys the whole illusion.

The dialogue system: a mini state machine inside the big one

Dialogue is its own little world. It has a queue of lines, a typewriter effect, and it freezes the game while it's up. Look at how `say()` works:

```
function say(lines, onDone) {
  dlg.lines = lines; dlg.idx = 0; dlg.onDone = onDone || null;
  state = 'dialogue';           // ← freezes movement (chapter 1's gate)
  showLine();
  document.getElementById('dialogue-box').classList.remove('hidden');
}
```

Passing `onDone` — a function to run *after* the last line — is that callback pattern again. It's how a quest can say three lines and *then* grant a reward, in order:

```
say([
  { name, text: 'Clear the dark woods for me. Take my old lighter.' },
], () => {           // this runs only after the player clicks through
  addItem('lighter');
  flags.quest = 1;
});
```

The typewriter effect is a `setInterval` revealing two characters at a time, playing a blip every few characters. Notice the *skip*: if you press interact while text is still typing, it dumps the full line instantly instead of advancing. That's a small courtesy that every text game needs, and it's a nice example of handling “the user is faster than the animation.”

The save system: serialize, don't snapshot

Saving is one of the most instructive functions in the codebase:

```
function saveGame() {
  if (!world || player.classId === 'none') return; // don't save a non-game
  localStorage.setItem(SAVE_KEY, JSON.stringify({
    player: { classId, xp, level, gold, health, maxHealth, atk, def, spd,
              inventory: player.inventory,
              weapon: Object.keys(DATA.weapons).find(k => DATA.weapons[k] === player.currentWeapon) },
    flags,
    map: world.id,
    x: Math.round(player.x / TILE), y: Math.round(player.y / TILE)
  }));
}
```

Several real lessons packed in here:

- **You save the minimum to *reconstruct*, not a photograph of memory.** We don't save the entire world object (its tile grid, its sprites — huge, and rebuildable). We save the map's *id* and the player's *tile* coordinates. On load, `loadMap('city', x, y)` rebuilds everything from that. **Store the seed, not the tree.**
- **JSON.stringify needs plain data.** `player.currentWeapon` is a *reference* to a weapon object — that can't be meaningfully saved. So we convert it to its key (`'rifle'`) with that `Object.keys(...).find(...)` line, and on load we look the key back up. Turning live references into stable ids for storage, and back

again, is called **serialization**, and it's a daily task in real backends (saving to a database is the same problem).

- **The guard clause** `if (!world ...) return;` refuses to save nonsense (like the title screen). Guarding the entry of a function against states it can't handle keeps the rest of the function simple.

`loadGame()` is the mirror image: parse the JSON, copy fields back onto `player` and `flags`, re-resolve the weapon key, and `loadMap` to the saved position. Save/load being symmetric mirror functions is a sign you've drawn the boundary well.

Input handling: normalize early

```
function onKeyDown(ev) {
  const k = ev.key.length === 1 ? ev.key.toLowerCase() : ev.key;
  // ...now everything downstream compares against a clean `k`...
}
```

Browsers report 'A' vs 'a' depending on Shift, and multi-character names like 'ArrowUp'. Rather than handle every case everywhere, we **normalize once at the entrance** — lowercase single letters, leave named keys alone. Downstream code stays simple because the mess was cleaned up at the door. “Normalize input at the boundary, keep the core clean” is a principle that scales from keypresses to API request parsing.

combat.js — a self-contained subsystem

`combat.js` is worth studying as a model of a **cohesive module**: everything about fighting is here and nothing else is. Its public surface is tiny — `start()`, `bindUI()`, `handleKey()`, `calculateDamage()` — and everything else is private.

The damage formula: preserved from the original game

```
function calculateDamage(attacker, defender) {
  const w = attacker.currentWeapon;
  const weaponDamage = w.dmg + (w.dex / 2) + (w.rng / 3);
  let baseDamage = (attacker.atk - defender.def) * 2 + weaponDamage;
  if (attacker.spd > defender.spd) baseDamage += attacker.spd / 3;
  const randomDamage = (Math.floor(Math.random() * 10) + 1) + baseDamage;
  return Math.floor(Math.max(8, randomDamage));
}
```

This function is a whole design in miniature. Read what it says: your weapon's dexterity and range matter less than its raw damage (they're divided down); attack fights defense (and is doubled, so stats swing hard); being faster than your foe adds a bonus; and there's a random 1–10 kicker so fights aren't perfectly deterministic. The final `Math.max(8, ...)` guarantees every hit does *something* so battles can't stall.

The lesson isn't the arithmetic — it's that **game balance lives in one readable function**. All the “feel” of combat is here, tunable, in ten lines. When you build systems, try to concentrate the tunable “policy” into small, named, findable places rather than smearing magic numbers across the codebase.

The turn loop: driven by `setTimeout`, gated by `busy`

Combat is turn-based, but it *animates*, so turns can't happen instantly — the muzzle flash, the hit, the enemy's counterattack all need to breathe. The code uses `setTimeout` to space events out in time, and a `busy` flag to lock the buttons while an animation plays:

```
function playerAttack() {
  if (busy || !active) return; // ignore clicks mid-animation
  setButtons(false);          // lock the UI
  AudioSys.sfx.fire(player.currentWeapon.sound);
  setTimeout(() => {
    const dmg = calculateDamage(player, enemy);
    enemy.health = Math.max(0, enemy.health - dmg);
    // ...flash, shake, sound, update the bar...
    setTimeout(() => {
      if (enemy.health <= 0) winSequence();
      else enemyTurn(); // hand the turn to the enemy
    }, 900);
  }, 200);
}
```

The `if (busy)` return guard at the top is doing important defensive work: without it, a player mashing the attack button would fire five overlapping turns and corrupt the fight. **Guarding against re-entry while an async operation is in flight** is a bug class you'll fight for your whole career (double-submitted forms, double-charged payments). Here it's one boolean, checked at every entry point.

Rendering the battle: a second independent loop

Combat runs its *own* `requestAnimationFrame` loop (render) while it's active, separate from the world loop. It draws the backdrop, the enemy sprite bobbing, and layers on timed effects — a white hit-flash, a screen shake, a red “you got hurt” wash — each controlled by a little “until” timestamp:

```
if (ts < anim.shakeUntil) x += Math.sin(ts / 18) * 6; // shake if we're still within the window
```

The pattern — “do this effect until time T ” — is a clean way to fire-and-forget an animation without tracking it frame by frame. You set `anim.shakeUntil = now + 300` and the render loop naturally stops shaking 300ms later. Time-based, not frame-counted, so it's frame-rate independent just like movement.

maps.js — a tiny builder DSL

`maps.js` shows off a different technique: a small **builder** helper that makes describing a map readable.

```
class Builder {
  constructor(w, h, base) {
    this.g = Array.from({ length: h }, () => Array(w).fill(base)); // a grid full of `base`
  }
  rect(x, y, w, h, id) { /* fill a rectangle of the grid with tile `id` */ }
  building(x, y, w, h, rng) { /* roof + a wall face, with some vines */ }
  scatter(id, density, rng) { /* randomly sprinkle a tile */ }
}
```

With that vocabulary, a whole area reads almost like English:

```
const b = new Builder(38, 26, 'grass');
b.border('fence'); // wrap the map in fence
b.rect(1, 12, 30, 2, 'dirt'); // main street
b.building(10, 3, 10, 5, r); // the elder's hall
b.rect(17, 10, 2, 2, 'water'); // the well
```

This is a **domain-specific language (DSL)** in miniature — a small set of building blocks tuned to one problem (drawing tile maps) so the actual content is short and clear. You didn't need a fancy library; a class with four

good methods gave you a readable way to express dozens of maps. When you notice yourself writing the same low-level pattern over and over, **build the vocabulary that makes it disappear**.

Each map is also a plain data object describing its portals, props, chests, enemies, and triggers:

```
portals: [
  { x: 43, y: 16, w: 1, h: 4, map: 'road', tx: 1, ty: 6, name: 'THE LONELY ROAD' },
  { x: 19, y: 0, w: 5, h: 1, map: 'city', tx: 21, ty: 27,
    cond: () => Game.flags.quest >= 2, // ← a locked exit
    msg: 'The dead city lies north...' }
]
```

That `cond` is elegant: a portal can carry a *function* that decides whether it's open yet. The engine just checks `if (p.cond && !p.cond()) { ...block... }`. Story gating — “you can't go north until quest 2” — becomes **data**, not a special case buried in engine code. New gated area? Add a portal with a `cond`. This is the data-driven philosophy from chapter 1 paying off concretely.

Cross-cutting habits you should steal

Zooming out, notice the small disciplines that repeat across every file:

- **Guard clauses first.** Functions bail out early on bad/irrelevant state (`if (busy) return; if (!world) return;`) so the main logic runs unindented and unworried.
- **Name your constants.** `TILE = 16`, `VIEW_W = 480`. The number 16 appears conceptually everywhere, but you can change the tile size in *one* place.
- **Reference by id, resolve late.** Weapons, sprites, sounds, maps — all referred to by string name and looked up when needed, never hard-wired.
- **One responsibility per function.** `calculateDamage` only calculates damage. `saveGame` only saves. `blocked` only answers “is this spot solid?” Small, single-purpose functions compose into big behavior and stay easy to reason about.
- **Time, not frames.** Everything that moves or animates is scaled by real time, so the game behaves identically on any hardware.

None of these are clever. That's the point. **Most good code is not clever — it's a hundred small, boring, consistent decisions** that add up to something you can still understand six months later.

Next: 03 — Graphics & Audio, the “wait, there are no image or sound files?” chapter — how pixels and sound are conjured from pure code.

03 — Graphics & Audio (from nothing but code)

Open the project folder. Notice what isn't there: no .png sprites, no .wav sound effects, no .mp3 music. Yet the game has pixel-art characters, animated combat, a gunshot for every weapon, and an ambient wind-and-drone soundtrack. **Every pixel and every sound is generated by math at load time.** This chapter is how — and why that's a genuinely useful thing to understand even if you never make a game.

Part 1 — Graphics: drawing pixels with code

The Canvas: a programmable sheet of graph paper

The browser gives us `<canvas>`, a rectangle of pixels you draw on with JavaScript. The whole game world renders onto one canvas defined in `index.html`:

```
<canvas id="game-canvas" width="480" height="320"></canvas>
```

That's a **480×320 grid of pixels**. Small on purpose — it's a *low-resolution* canvas that CSS then stretches up to fill the screen. This is the secret to the crisp pixel-art look. Two settings make it work:

```
#game-canvas { image-rendering: pixelated; } /* CSS: scale up with hard edges, no blur */
```

```
ctx.imageSmoothingEnabled = false; /* JS: don't blur when drawing images */
```

Without these, scaling a 16×16 sprite up to 60×60 would blur it into mush. With them, each original pixel becomes a crisp block. **We draw tiny and display big.** That single idea is why the art looks intentionally retro instead of accidentally low-quality.

Turning text into sprites

Here's the trick that makes hand-drawing sprites in code bearable. In `sprites.js`, the `px()` function reads a sprite as **ASCII art** — a list of strings where each character is a pixel — plus a palette that maps characters to colors:

```
function px(rows, pal, w) {
  const c = cv(width, rows.length);
  const ctx = c.getContext('2d');
  rows.forEach((row, y) => {
    for (let x = 0; x < row.length; x++) {
      const col = pal[row[x]]; // look up this character's color
      if (col) { ctx.fillStyle = col; ctx.fillRect(x, y, 1, 1); } // paint one pixel
    }
  });
  return c; // hand back a ready-to-use mini-canvas
}
```

So a medkit icon is *literally drawn in the source code* as a picture you can see:

```
store.icon_medkit = px([
  '.....',
  '...oooooooo...',
  '..oWWWWWWWWo..',
  '..oWWWRWWWWo..', // o = outline, W = white, R = red
  '..oWWWRWWWWo..',
  '..oWRRRRRRWWo..', // the red cross emerges from the letters
  '..oWRRRRRRWWo..',
])
```

```
'..oWWWWRRWWWo..',
'..oWWWWRRWWWo..',
'..oWWWWWWWWWo..',
'...ooooooooo...'
], { o: '#17130e', W: '#d8d5c8', R: '#b3372e' });
```

You can *read the picture in the code*. This is a beautiful example of choosing a **representation that fits the human**, not just the machine. The computer would be equally happy with an array of numbers; a person editing the art would not. Picking data formats that humans can read and edit is a real, underrated engineering skill.

Each sprite is rendered onto its own small off-screen <canvas> once, at startup, and stored in a lookup table (store['icon_medkit']). During the game we just drawImage those pre-made canvases — fast, because the drawing-from-strings work happened once. This is **caching / precomputation**: do expensive work once up front, reuse the result cheaply forever. (toURL() even converts a sprite to a data-URL so the HTML tags in the shop and inventory can display the same generated art.)

One character, eight animations, from six pictures

Drawing every direction and frame of every character by hand would be brutal. Instead, the code is lazy in the smart way. buildHuman() defines a character in just a few poses (down, up, side, each with two walk frames) and then **generates the rest**:

```
if (dir === 'side') {
  store['${name}_right_${f}'] = c;
  store['${name}_left_${f}'] = flipH(c); // left is just right, mirrored
}
```

flipH() mirrors a canvas horizontally, so “walking left” is free — it’s “walking right” flipped. And every character class (Survivalist, Raider, etc.) reuses the *same body shape* with a different color palette:

```
buildHuman('player_survivalist', { J: '#6b5a3a', K: '#52432a', h: '#4a3b28' }); // jacket/shade/hair
buildHuman('player_raider',      { J: '#7a3b2e', K: '#5c2a20', h: '#2e2a26' });
```

One template, many skins. This is **Don’t Repeat Yourself (DRY)** in art form: define the shape once, vary only what differs (the colors). When you catch yourself about to copy-paste-and-tweak, ask instead: *what’s the template, and what’s the parameter?*

Procedural tiles and the seeded random number generator

The world is built from tiles — grass, cracked road, rubble, pines. If every grass tile were identical, the ground would look like graph paper. So tiles have **variants** with randomly sprinkled specks and blades of grass. But here’s the problem: if we used plain Math.random(), the world would look *different every single frame*, shimmering like TV static.

The fix is a **seeded pseudo-random number generator** — mulberry32:

```
function mulberry32(a) {
  return function () {
    a |= 0; a = (a + 0x6D2B79F5) | 0;
    let t = Math.imul(a ^ (a >> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >> 7), 61 | t)) ^ t;
    return ((t ^ (t >> 14)) >> 0) / 4294967296; // a number in [0, 1)
  };
}
```

You don’t need to follow the bit-twiddling. What matters is the *property*: given the same starting **seed** number, it produces the exact same sequence of “random” numbers every time. So:

```
const rng = mulberry32(101); // seed 101 → always the same wasteland
```

The wasteland is randomly generated, but it's the *same* random wasteland on every load, because it's built from a fixed seed. This is the concept behind Minecraft world seeds, procedural game levels, and — importantly — **reproducibility** in serious software. A “random” test that uses a seeded RNG fails the same way every time, so you can actually debug it. *Controlled* randomness — random-looking but repeatable — is one of the most useful tools you can have.

Real distinction to carry: `Math.random()` is for things that should differ every time (falling ash, damage rolls). A *seeded* RNG is for things that should look varied but stay *stable* (the map). Choosing which kind of randomness you want is a real decision, and mixing them up is a classic bug.

Scenes: gradients, silhouettes, and layered depth

The bigger backdrops — the title skyline, combat arenas, the intro's mushroom cloud — are painted with gradients and simple shapes, layered back to front like stage scenery. `drawBackdrop()` for the sunset “wastes,” for instance:

1. a vertical **gradient** for the sick orange sky,
2. a low sun,
3. a **silhouette skyline** of broken buildings (generated with a seeded rng so it's stable),
4. dead trees in front,
5. a dark ground band,
6. and finally **ash motes** sprinkled on top.

Each layer is just rectangles and gradients, but stacked in the right order they read as a place with depth. That “background → midground → foreground, drawn in that order” is the painter's algorithm again (you met it in chapter 2 for sprites) — the same idea that makes the whole thing feel three-dimensional on a flat canvas.

Lighting: darkness as a layer you subtract from

The dark maps (woods, factory interior) do something clever with **compositing**. To make a pool of light around the player, the code covers the screen in near-black, then *erases a soft circle* out of that darkness where the player stands:

```
d.fillStyle = 'rgba(2, 4, 8, 0.93)';
d.fillRect(0, 0, VIEW_W, VIEW_H); // 1. cover everything in dark

const g = d.createRadialGradient(px, py, r*0.2, px, py, r); // 2. a soft circle
g.addColorStop(0, 'rgba(0,0,0,1)'); // solid in the middle...
g.addColorStop(1, 'rgba(0,0,0,0)'); // ...fading to nothing at the edge

d.globalCompositeOperation = 'destination-out'; // 3. "erase" mode
d.fillStyle = g;
d.fillRect(0, 0, VIEW_W, VIEW_H); // punch the light-hole in the dark
```

`destination-out` means “wherever I paint, *remove* what's already there.” So painting the gradient doesn't add light — it deletes darkness, leaving a soft-edged window onto the world. Carrying a lighter simply makes the erased radius bigger ($r = 95$ vs 48). **Compositing modes** — rules for how new paint combines with existing paint — are a whole toolbox; this is one small, striking use of them.

Part 2 — Audio: sound as pure math

There isn't a single sound file. Every effect is **synthesized** — built from raw waveforms — using the **Web Audio API** in `audio.js`. To get why that's possible, you need one idea: **sound is just a wiggling number over time**. A speaker cone pushed in and out fast makes sound; describe the push-and-pull as numbers and you've described a sound.

The two atoms: tones and noise

Almost every effect in the game is built from two primitive generators.

A **tone** is a pure pitch — a smooth oscillating wave at some frequency:

```
function tone(freq, dur, { type = 'square', vol = 0.2, slideTo = null } = {}) {
  const osc = ctx.createOscillator(); // a wave generator
  const g = ctx.createGain(); // a volume knob
  osc.type = type; // square / sawtooth / triangle / sine
  osc.frequency.setValueAtTime(freq, t);
  if (slideTo) osc.frequency.exponentialRampToValueAtTime(slideTo, t + dur); // pitch slide
  g.gain.linearRampToValueAtTime(vol, t + 0.008); // quick fade-in (attack)
  g.gain.exponentialRampToValueAtTime(0.0001, t + dur); // fade-out (decay)
  osc.connect(g).connect(master);
  osc.start(t); osc.stop(t + dur + 0.05);
}
```

Two things to notice. First, the **waveform type** changes the character: a square wave sounds retro and buzzy (classic 8-bit), a triangle softer, a sawtooth harsh. Second, that **envelope** — the quick fade-in then fade-out on the gain (volume) — is what makes a note sound like a *pluck* or *beep* instead of an ugly click. Real instruments have this “attack and decay” shape; faking it is what makes synth sounds feel like sounds.

Noise is the opposite — all frequencies at once, like static or wind. It's made once as two seconds of random samples and reused:

```
const d = noiseBuf.getChannelData(0);
for (let i = 0; i < len; i++) d[i] = Math.random() * 2 - 1; // random values in [-1, 1] = hiss
```

Then a **filter** shapes that hiss into something specific — a low rumble, a sharp “tss,” wind — by only letting certain frequencies through. Noise + filter is the backbone of every percussive and textural sound in the game.

Building recognizable effects from the atoms

Now the fun part: combine tones and noise and you get *characterful* sounds. Each weapon fires differently, and you can read the sound design right in the code:

```
fire(kind) {
  switch (kind) {
    case 'rifle': // one sharp crack
      noise(0.28, { vol: 0.4, freq: 2200, slideTo: 150 }); // bright burst dropping fast
      tone(120, 0.2, { type: 'sawtooth', slideTo: 50 }); // low thump underneath
      break;
    case 'shotgun': // bigger, boomier
      noise(0.4, { vol: 0.5, freq: 1400, slideTo: 90 });
      tone(90, 0.3, { type: 'sawtooth', slideTo: 40 });
      break;
    case 'smg': // three fast pops
      for (let i = 0; i < 3; i++)
        noise(0.09, { vol: 0.3, freq: 1900, delay: i * 0.09 }); // staggered in time
```

```

    break;
  }
}

```

A gunshot = a bright filtered-noise burst (the crack) layered over a low sawtooth thump (the punch), with the pitch sliding *down* fast to mimic the report dying. The SMG is literally a `for` loop firing three quiet cracks 90ms apart. **You are composing sound the way you'd compose a function** — small parts, combined and timed. Once you see that a “gunshot” is just “noise burst + low thump, pitch falling,” synthesis stops being mysterious.

The soundtrack: layers that never repeat

The ambient music isn't a looping track — it's **three generated layers** running together, so it never audibly repeats:

1. **Wind** — filtered noise whose filter frequency is slowly swept up and down by a very slow oscillator (an **LFO**, low-frequency oscillator), so it breathes.
2. **A drone** — two low triangle waves tuned *slightly* apart (55 and 55.6 Hz). That tiny mismatch makes them slowly “beat” against each other, a shimmering detuned pad. Deliberate imperfection sounding better than perfection.
3. **Sparse melody notes** — every couple of seconds, *maybe* (35% of the time it stays silent) a single note from a minor scale, fed through an **echo** (a delay line with feedback) so each note repeats and fades into the cold.

```

const SCALE = [73.42, 87.31, 98.0, 110.0, 130.81, 146.83]; // a bleak minor scale
melodyTimer = setInterval(() => {
  if (muted || Math.random() < 0.35) return; // sometimes play nothing – space is part of it
  const f = SCALE[Math.floor(Math.random() * SCALE.length)];
  // ...play a soft triangle note through the echo...
}, 2600);

```

Because pitches are *chosen* from a scale but *timed* by chance, the result is always musical but never the same twice — endless, non-repeating, and a few kilobytes of code instead of a multi-megabyte audio file. Generative music from a scale plus randomness is a lovely little demonstration of **constraints + chance = something that feels composed**.

One volume knob to rule them all: the master gain

Every sound connects through a single master gain node before reaching the speakers:

```

osc.connect(g).connect(master); // every effect routes through `master`
master.connect(ctx.destination); // master → speakers

```

That's why muting is one line — turn down `master` and *everything* goes quiet at once, no matter how many sounds are playing:

```

function toggleMute() { muted = !muted; master.gain.value = muted ? 0 : 0.5; }

```

This is a **single point of control**, and it's the audio version of a system-design principle you'll use forever: route related things through one chokepoint so you can control them all at once. (It's the same reason apps funnel every network call through one client, or every log line through one logger — control the flow at one place.)

Why generate assets instead of loading files?

It's worth being honest about the trade-off, because “generate everything” is *not* always the right call.

What it buys you here: - **Zero external files** → the whole game is a handful of tiny text files. Loads instantly, works offline, nothing to 404. - **Everything is version-controlled and diffable.** A change to the art shows up as a code diff you can review, not an opaque binary blob. - **Infinitely tweakable.** Want the rifle punchier? Change a number. No art tool, no re-export.

What it costs: - **A real artist is faster in a real art tool.** Hand-drawing a 24×24 bear in ASCII is slower than painting it in Aseprite for someone skilled. - **There's a ceiling on detail.** This approach shines for stylized pixel art; it would be absurd for photorealistic graphics or orchestral music.

So the rule isn't “always generate.” It's the same calibration lesson from chapter 1: **match the technique to the project.** For a small, stylized, dependency-free game made by one person who's also learning, generating everything is a fantastic fit — and it happens to teach you how pixels and sound *actually work* underneath the tools, which is knowledge the tools normally hide from you.

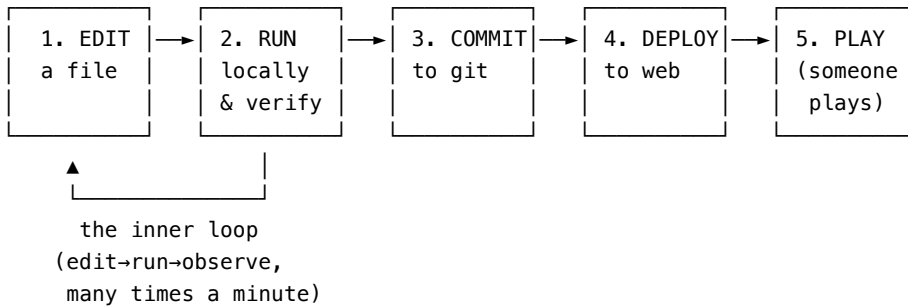
Next: 04 — DevOps — how this code gets from your editor onto the internet where someone can actually play it.

04 — DevOps

“DevOps” sounds like a job title with a Kubernetes cluster attached. Strip away the buzzwords and it’s a simple question: **how does code get from your editor into someone’s hands, reliably and repeatably?** Every project answers it, whether or not anyone calls it DevOps. This chapter walks Nuclear Winter’s answer — which is deliberately minimal — and then shows you the next rungs of the ladder so you know where you’re headed.

1. The lifecycle of a change

Here’s the full journey of one edit to this game, from thought to player:



Steps 1–2 are the **inner loop** — the thing you do hundreds of times a day. Steps 3–5 are the **outer loop** — sharing and shipping. DevOps is mostly about making both loops *fast* and *trustworthy*. A slow inner loop wastes your life in ten-second increments; a flaky outer loop means “works on my machine” and 2 a.m. panics. Let’s walk each step.

2. The inner loop: edit → run → observe

Running the game locally

The top-level **README** says it:

```
python3 -m http.server 8642
# then open http://localhost:8642
```

Why a server at all? You *could* double-click `index.html` and it’d mostly work — but loading over `file://` breaks some browser features (fonts, certain fetches, and later if you switch to ES modules, those too). Serving over `http://` mirrors how the game will really run on the web, so **your local environment matches production**. That “dev matches prod” instinct — test in conditions that resemble reality — prevents a whole category of “worked locally, broke live” surprises. `python3 -m http.server` is just the smallest possible web server; any static server (`npm run serve`, VS Code’s Live Server, etc.) does the same job.

The `.claude/launch.json` file

This repo has a small config at `.claude/launch.json`:

```
{
  "version": "0.0.1",
  "configurations": [
    {
      "name": "nuclear-winter",
      "runtimeExecutable": "python3",
      "runtimeArgs": ["-m", "http.server", "8642"],
      "port": 8642
    }
  ]
}
```



```
]
}
```

It's a tiny piece of “**configuration as code**”: instead of everyone remembering the exact command to start the dev server, the command is *written down in the repo*. New contributor clones the project and the “how do I run this” answer travels *with* the code. This is the germ of a huge DevOps idea — **encode operational knowledge in files, not in people's heads or a wiki that goes stale**. The same instinct scales up to Dockerfiles, Makefiles, CI configs, and infrastructure-as-code.

Observing: your browser's DevTools are your instruments

The “observe” half of the inner loop matters as much as running. When something's wrong, you don't guess — you look. Open the browser DevTools (F12) and you have:

- **Console** — where `console.log()` output and errors show up. If the game freezes, an error here usually says exactly which file and line died.
- **Network tab** — did all six `js/` files actually load? A red 404 here explains a lot of “nothing happens” mysteries.
- **Sources / debugger** — you can set a *breakpoint* on a line and the game pauses there so you can inspect variables. Far more powerful than sprinkling `console.log`.

Learning to *read your instruments* instead of guessing is one of the biggest force multipliers in engineering. A junior dev stares at code wondering why; a stronger dev opens the console, reads the error, and knows in five seconds.

3. The outer loop: version control with Git

What Git actually is, and why every project uses it

Git is a **time machine and a collaboration tool** for your code. It records snapshots (“commits”) of your project, each with a message explaining *why* the change happened. This buys you three things you can't live without:

1. **Undo across time.** Broke something three days ago? You can see exactly what changed and revert it. Your code has a rewindable history.
2. **A written record of why.** `git log` is a narrated story of how the project got to now. Good commit messages are documentation that never goes out of date because it's attached to the change itself.
3. **Safe parallel work.** *Branches* let you build a risky feature off to the side without endangering the working version, then merge it in when it's ready.

The everyday Git rhythm

```
git status          # what have I changed? (run this constantly)
git diff            # show me exactly what changed, line by line
git add js/game.js  # stage the files I want in this commit
git commit -m "Add a new enemy type: the Ash Wraith" # snapshot, with a reason
git log --oneline    # the project's history, one line per commit
```

A few habits that separate tidy repos from messy ones:

- **Commit small and often.** One commit should be one coherent idea (“add the shop,” “fix the flee bug”), not “did a bunch of stuff.” Small commits are easy to review, easy to revert, and easy to understand later.

- **Write the message for future-you.** "fix" tells you nothing in six months. "Fix flee button letting you escape the boss fight" tells the whole story. Explain *why*, not just *what* — the diff already shows what.
- **Branch for anything risky.** `git switch -c try-daynight-cycle` gives you a sandbox. If it works, merge it; if it doesn't, throw the branch away and nothing was harmed.

What *not* to commit: the `.gitignore`

Not everything belongs in version control. Generated files, secrets, OS junk (`.DS_Store`), and dependency folders (`node_modules`) should be ignored. This project barely needs one because it has no build output and no dependencies — but the moment you add tooling, create a `.gitignore`:

```
# example, for when this project grows
node_modules/
dist/
.DS_Store
*.log
.env          # NEVER commit secrets/API keys — this is the big one
```

Burn this in now: never, ever commit passwords, API keys, or tokens. Git remembers *everything* — deleting a secret in a later commit doesn't remove it from history. Leaked keys in git history are one of the most common real-world security incidents. Secrets go in environment variables or a secrets manager, never in the repo.

4. Deploying: getting it onto the internet

Because this is a **static site** — just HTML, CSS, and JS with no server-side code — deploying is gloriously easy. There's nothing to "run" on a server; a host just needs to hand your files to browsers. That makes it perfect for free static hosting.

The easiest real deployment: GitHub Pages

1. Push the repo to GitHub.
2. In the repo's **Settings** → **Pages**, choose the `main` branch as the source.
3. GitHub serves it at `https://<your-username>.github.io/nuclear-winter/`.

That's a real, public, HTTPS-secured deployment of your game, for free, in three clicks. Netlify, Vercel, and Cloudflare Pages do the same thing with similar ease and add niceties like automatic deploys on every push. The reason it's *this* easy is directly downstream of a system-design choice from chapter 1: **no backend means no servers to manage**. Architecture decisions and ops difficulty are the same coin — a simpler system is a simpler thing to run. That connection is the whole reason "DevOps" exists as a discipline: the people building it and the people running it are making one set of decisions together.

The one gotcha: relative paths

Notice the game loads its scripts with **relative** paths:

```
<script src="js/game.js"></script>  <!-- not "/js/game.js" -->
```

That leading-slash-or-not matters. `/js/game.js` means "from the root of the domain," which breaks when your game lives in a *subfolder* like `username.github.io/nuclear-winter/`. `js/game.js` means "relative to this page," which works no matter what folder it's deployed into. Small detail, extremely common first-deploy bug ("it works locally but all my files 404 on GitHub Pages"). The fix is almost always: use relative paths.

5. Automation: CI/CD, and a starter pipeline

Right now the outer loop is manual: you commit, you deploy. **CI/CD** — Continuous Integration / Continuous Deployment — is about letting a robot do the repetitive, error-prone parts *every single time*, so quality doesn't depend on you remembering.

- **CI (Continuous Integration)**: every time you push, automatically *check* the code — run tests, run a linter, make sure it isn't broken. Catch problems in minutes, before they reach anyone.
- **CD (Continuous Deployment)**: every time the checks pass on `main`, automatically *deploy* it. No human runs a manual deploy; merging *is* shipping.

Here's a real, working GitHub Actions workflow you could add at `.github/workflows/deploy.yml`. It checks the JavaScript for syntax errors on every push and deploys to GitHub Pages when `main` is green:

```
name: CI + Deploy
on:
  push:
    branches: [main]

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # get the code
      - uses: actions/setup-node@v4        # install Node (for the syntax check)
        with: { node-version: '20' }
      - name: Syntax-check every JS file   # the "CI" gate
        run: for f in js/*.js; do node --check "$f"; done

  deploy:
    needs: check                          # only deploy if `check` passed
    runs-on: ubuntu-latest
    permissions: { pages: write, id-token: write }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/upload-pages-artifact@v3
        with: { path: '.' }
      - uses: actions/deploy-pages@v4      # the "CD" step
```

Read the *shape*, not the exact syntax. A pipeline is a list of **jobs**, each a list of **steps**, triggered by an **event** (here, “push to main”). The `needs: check` line is the key idea: **deploy is gated on checks passing**. The robot refuses to ship broken code. That gate is the entire value of CI — it turns “I hope I didn't break anything” into “the machine confirmed I didn't.”

Notice the CI step reuses the *exact* command you'd run by hand — `node --check`. Good automation is usually just “the manual thing you already do, run automatically and consistently.” You don't need to master YAML today; you need to understand that CI is a **safety net that runs the same checks every time so humans can't forget**.

What you'd add as the project grows

The moment this game gets more serious, the natural next rungs on the ladder:

Need	Tool you'd reach for	What it does
Catch style/bug patterns	ESLint	Flags likely mistakes and enforces consistency.

Need	Tool you'd reach for	What it does
Consistent formatting	Prettier	Auto-formats so nobody argues about spaces.
Confidence when refactoring	Vitest / Jest	Automated tests that prove logic (like <code>calculateDamage</code>) still works.
Faster loads, bundling	Vite / esbuild	Bundles + minifies; enables ES modules.
Types that catch bugs early	TypeScript	Catches “you passed a string where a number goes” before runtime.

Each of these is a rung, not a requirement. Add them *when the pain they solve is real*, not before. A test suite for a 200-line game nobody's shipping is procrastination dressed as diligence. A test suite for `calculateDamage` once six people depend on the balance being stable is wisdom. Same tool, different moment. **The judgment of *when* is the actual skill** — the tools themselves are easy to learn once you need them.

6. The mental model to keep

Everything in this chapter reduces to one loop, repeated at two speeds:

make a change → verify it's good → share it → repeat

- The **inner loop** (change → verify) should be *seconds*. Optimize it ruthlessly — it's most of your life as a developer.
- The **outer loop** (share → ship) should be *trustworthy*. Automate the checks so “good” doesn't depend on anyone being careful on a bad day.

DevOps, at every scale from this tiny game to a company running thousands of servers, is just making those two loops fast and reliable. The tools get fancier; the goal never changes.

Next: 05 — Exercises — because you don't actually learn any of this by reading. You learn it by changing the code and watching what happens.

05 — Exercises & Extension Roadmap

Reading about code is like reading about swimming. Useful up to a point, then you have to get in the water. This chapter is the water. The exercises are ordered from “change one number” to “design a whole subsystem,” and each one names the skill it’s secretly training. Do them in order — each assumes you’ve felt the ones before it.

The one rule: after every change, *run the game and look*. Edit → refresh → observe. If it broke, open the browser console (F12) and read the error before changing anything else. Diagnosing beats guessing, every time.

Tier 0 — Orientation (find your footing)

These build the muscle of *navigating* a codebase, which is 90% of real work.

1. **Find the number.** The player walks at a certain speed. Find where that speed is set in `js/game.js` (hint: search the file for `spd =` or the comment “pixels”). Change it, refresh, feel the difference. Put it back. *Skill: locating behavior in unfamiliar code.*
 2. **Follow a string.** In `js/data.js` the Hunting Rifle has `sound: 'rifle'`. Trace that string: where does 'rifle' get used? (Search `sound` in `combat.js`, then `fire()` in `audio.js`.) You’ve just followed data across three files. *Skill: tracing how loosely-coupled systems connect.*
 3. **Read the state machine live.** Open the console and type `Game.flags` while playing, then `Game.player`. Watch them change as you play. You’re inspecting live program state — exactly what a debugger does. *Skill: observing runtime state.*
-

Tier 1 — Tuning (change data, not logic)

This whole tier is editing `js/data.js` — no engine code. That’s the point: **good architecture lets you change the game without touching the machine.**

4. **Rebalance a weapon.** Make the Revolver hit harder — bump its `dmg`. Fight something and confirm. Now ask: did you change any logic? (No. You changed a fact, and `calculateDamage` picked it up.) *Skill: feeling data-driven design from the inside.*
 5. **Add a brand-new weapon.** Add an entry to `weapons`, give it an existing `icon` and `sound`, then add it to `shopStock` so you can buy it. Play, buy it, wield it. You added content with zero new logic. *Skill: extending a system through its data, not its code.*
 6. **Invent an enemy.** Add an entry to `enemyTypes` reusing an existing `sprite`. Then place it in a map: in `js/maps.js`, add it to some area’s `enemies` array. Walk into it. *Skill: understanding how content (data) and placement (maps) combine.*
 7. **Write a new sign.** Find a `sign` prop in a map’s `props` array and change its `text`, or add a new one. Read your words in-game. *Skill: the content-authoring loop.*
-

Tier 2 — Small logic changes (touch the engine gently)

Now you edit behavior, but in contained, low-risk spots.

8. **Make medkits heal a percentage.** Medkits currently heal a flat amount. Find where (search `heal` in `game.js` and `combat.js`) and change it to heal, say, 40% of max HP instead. *Skill: locating and safely modifying a single behavior that lives in two places — and noticing it lives in two places.*

9. **Add a new pixel-art icon.** In `js/sprites.js`, copy the `icon_medkit` block, rename it, and redraw the ASCII grid into something new (a canteen? a grenade?). Point a new item at it in `data.js`. *Skill: editing generated art; seeing that “art” here is just data you can author.*
 10. **Design a new sound.** In `js/audio.js`, add a case to the `fire()` switch (or a new `sfx` entry) that layers a tone and a noise. Wire a weapon’s sound to it. *Skill: composing an effect from primitives; hearing the tones-and-noise model from chapter 3.*
 11. **Tune the fear.** In a dark map, the light radius without a lighter is 48. Find it in `game.js` (`renderDarkness`) and make the darkness tighter or looser. *Skill: connecting a single constant to the whole feel of an area.*
-

Tier 3 — New features (design a little)

These require adding something that wasn’t there. Sketch your plan on paper first — *what states change, what gets drawn, what the player does* — before you type.

12. **A stamina/sprint mechanic.** Hold Shift to move faster but drain a stamina bar; walking refills it. You’ll touch input (`onKeyDown`), the update loop (movement + drain), the player object (a `stamina` field), and the HUD (a new bar). *Skill: threading one feature through every layer — the most realistic exercise here.*
 13. **A second boss.** Add an `enemyTypes` entry with `boss: true`, draw it a unique sprite, place it in a map with a unique `id`, and add an `afterVictory` case in `game.js` so beating it advances the story. *Skill: following an existing pattern end-to-end (the Faction Leader is your template).*
 14. **A key-item puzzle.** Mirror the existing lighter-burns-webs or key-opens-door logic to gate a new area behind a new item. *Skill: recognizing and reusing an existing pattern instead of inventing a new one — a senior instinct.*
 15. **A settings toggle.** Add an on-screen button to toggle the CRT scanline effect (it’s a CSS `::after` in `styles.css`). *Skill: connecting a DOM control to a visual change; crossing the JS/CSS boundary.*
-

Tier 4 — Architecture & Ops (think like a builder)

Now the meta-skills. These change how the project is *built and shipped*, not what it does.

16. **Extract a module.** `game.js` is large. Pull all the shop code (`openShop` + related) into a new `js/shop.js` that exposes `Shop.open()`, and load it in `index.html` in the right order. Does the game still work? *Skill: drawing a new module boundary — and discovering the dependencies you didn’t know were there.*
 17. **Add real tests.** `calculateDamage` in `combat.js` is a pure function — perfect for testing. Set up Vitest, write tests asserting known inputs give expected ranges, and make them run. *Skill: the confidence-under-refactoring that automated tests buy.*
 18. **Ship it.** Push to GitHub, enable GitHub Pages, and get a public URL someone else can play. Fix the relative-path issues if they appear (chapter 4 warned you). *Skill: the full outer loop — code to live URL.*
 19. **Automate the check.** Add the CI workflow from **chapter 4** so every push syntax-checks your JS. Push a deliberately broken file and watch CI catch it, then fix it. *Skill: building a safety net and seeing it work.*
 20. **Migrate to ES modules.** The boss level. Convert the six IIFE files to real `import/export` modules and load with `<script type="module">`. You’ll need a bundler (Vite) or careful path handling. *Skill: understanding what the module pattern was faking, and what modern module systems actually give you.*
-

How to debug when it breaks (it will)

A repeatable method beats panic every time:

1. **Read the error.** Open the console (F12). An error names a *file and line*. Go there first. Half of all bugs are solved by actually reading the message instead of assuming.
2. **Reproduce it reliably.** Find the exact steps that trigger it. A bug you can summon on command is a bug you can fix; an intermittent one you can't reproduce is a nightmare. Make it reliable first.
3. **Narrow it down.** Add `console.log()` before and after the suspect line, or set a breakpoint. Confirm *what the values actually are*, versus what you assumed. Bugs live in the gap between those two.
4. **Change one thing.** Make a single change, test. If you change five things and it works, you don't know which fix mattered — and you may have added two new bugs.
5. **When truly stuck, explain it out loud.** Describe the problem, step by step, to a person or a rubber duck. You'll often catch the flaw mid-sentence. This is real; it's called *rubber-duck debugging* and professionals use it daily.

The difference between a beginner and a pro isn't that the pro writes bug-free code. Both write bugs constantly. The pro has a *method* for finding them, stays calm, and reads the evidence instead of guessing. That method is entirely learnable, and every bug you fix with it makes you faster at the next one.

Where to go after this codebase

You've now seen a complete, real program end to end. To keep growing:

- **Build your own tiny thing from scratch.** A different small game, a calculator, a to-do app. Starting from an empty file teaches what a finished codebase can't — you'll finally understand *why* each piece here exists, because you'll feel its absence.
- **Read code better than yours.** Find a small open-source project you admire and read it the outside-in way from chapter 2. Being able to *read* code is as important as writing it, and it's rarely taught.
- **Go deeper on whichever lens grabbed you.** Loved the graphics? Learn a real game engine or WebGL. Loved the structure? Read about design patterns and clean architecture. Loved the shipping? Learn Docker and cloud deployment. This game gave you a taste of all three so you could find out which one pulls at you.

The single most important habit, above every technique in these five documents: **keep the edit → run → observe loop tight, and keep it turning.** Everything else is detail.

Good luck out there in the wasteland.